

Aula 05

Aplicação das tabelas-verdade em exercícios

De conectivos isolados a fórmulas inteiras — e três rótulos que classificam qualquer uma delas.

tautologia · contradição · contingência

Nesta aula



Montando tabelas-verdade de fórmulas com vários conectivos

O mesmo processo de sempre, aplicado passo a passo, coluna por coluna.



Tautologia, contradição e contingência

Três rótulos que classificam qualquer fórmula lógica pelo seu padrão de valores.



Verificando tudo automaticamente

Gerando e testando cada combinação possível em C# (e em Python, para comparar).

ANTES DE COMEÇAR

Onde paramos na Aula 04

Vimos os cinco conectivos e a tabela-verdade de cada um, isoladamente.



“não”



“e”



“ou”



“se... então”



“se e somente se”

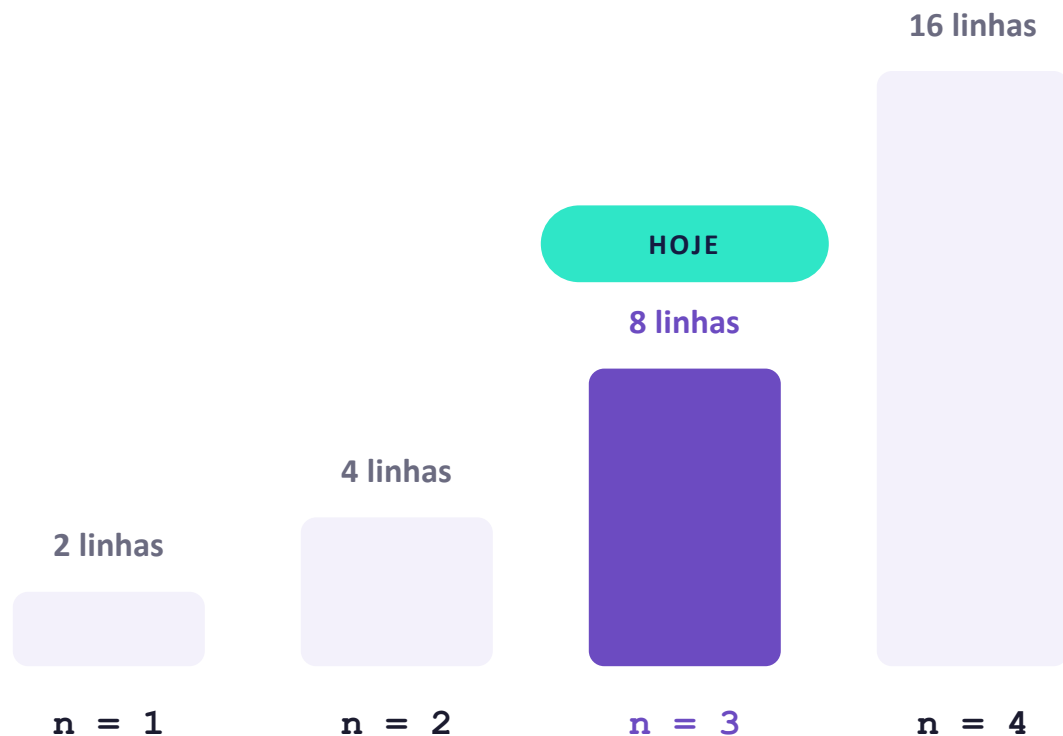


Hoje: combinar vários desses símbolos numa fórmula só, e dar nome ao que ela representa.



Fórmulas com múltiplos conectivos

Com n proposições simples, a tabela-verdade tem 2^n linhas — uma para cada combinação possível de valores.



Vamos montar a tabela de $(p \wedge q) \rightarrow r$ passo a passo (3 variáveis $\rightarrow$ 8 linhas).

A partir de 4-5 variáveis já é impraticável fazer à mão — é aí que o código entra, mais tarde nesta aula.

ANTES DE MONTAR A TABELA



Monte por colunas, não por linha

Preencha uma coluna auxiliar inteira — por exemplo, todo o $p \wedge q$ — antes de passar para a próxima. É mais fácil não errar do que calcular linha por linha, tentando fazer tudo de uma vez.

ordem sugerida: colunas de p , q , r → colunas auxiliares →
coluna final

PASSO A PASSO

Montando $(p \wedge q) \rightarrow r$

3 variáveis $\rightarrow$ 8 linhas. Primeiro as colunas de p , q , r ; depois a auxiliar $p \wedge q$; por último, a fórmula inteira.

p	q	r	$p \wedge q$	$(p \wedge q) \rightarrow r$
V	V	V	V	V
V	V	F	V	F
V	F	V	F	V
V	F	F	F	V
F	V	V	F	V
F	V	F	F	V
F	F	V	F	V
F	F	F	F	V

Só uma linha dá F: quando $p \wedge q$ é verdadeira mas a promessa (r) não se cumpre.

Tautologia, contradição e contingência

Depois de montar a coluna final, ela sempre se encaixa em um destes três padrões.



Tautologia

A fórmula é sempre verdadeira, para qualquer combinação de valores.



Contradição

A fórmula é sempre falsa, para qualquer combinação de valores.



Contingência

Depende dos valores — às vezes verdadeira, às vezes falsa.

Um exemplo de cada



$$p \vee \neg p$$

sempre V $\rightarrow$ tautologia

o princípio do terceiro excluído: “p ou não-p” nunca falha.



$$p \wedge \neg p$$

sempre F $\rightarrow$ contradição

não dá para p e não-p serem verdadeiras ao mesmo tempo.



$$p \rightarrow q$$

varia $\rightarrow$ contingência

o condicional que já vimos na Aula 04 — nem sempre V, nem sempre F.



Tautologias sustentam raciocínios válidos

Uma tautologia representa um raciocínio válido por estrutura, independente do assunto — não importa o que p e q signifiquem, ela nunca falha. É esse tipo de fórmula que sustenta regras de inferência confiáveis, como o modus ponens:

$$((p \rightarrow q) \wedge p) \rightarrow q$$

p	q	$p \rightarrow q$	$(p \rightarrow q) \wedge p$	$((p \rightarrow q) \wedge p) \rightarrow q$
V	V	V	V	V
V	F	F	F	V
F	V	V	F	V
F	F	V	F	V



A Lei de De Morgan

Monte lado a lado $\neg(p \wedge q)$ e $\neg p \vee \neg q$:

p	q	$\neg(p \wedge q)$	$\neg p \vee \neg q$
V	V	F	F
V	F	V	V
F	V	V	V
F	F	V	V

As duas colunas finais são idênticas — as fórmulas são logicamente equivalentes, mesmo parecendo diferentes escritas.



Tabela-verdade hoje, álgebra amanhã

De Morgan também pode ser provada manipulando símbolos, sem tabela nenhuma — é a mesma ideia de hoje, só que com outra ferramenta. Isso é o assunto da Aula 06.



“Se chove ou está frio, leve casaco”

p: “chove”

q: “está frio”

r: “levo casaco”

regra ideal: $(p \vee q) \rightarrow r$



Só que essa fórmula é uma contingência: ela pode dar V ou F dependendo se você realmente segue a recomendação (r) nas situações em que ela se aplica ($p \vee q$).

Diferente de uma tautologia, aqui o valor da fórmula depende de um fato do mundo real — você ter levado o casaco ou não — e não só da estrutura lógica.



Chove, mas você não leva casaco — a promessa falhou.



Não chove, mas está frio, e você leva casaco — cumpriu.



Nem chove, nem está frio — a recomendação não se aplica; vacuamente V.

Os três rótulos, lado a lado

Tipo	Coluna final	Exemplo	Como reconhecer
Tautologia	sempre V	$p \vee \neg p$	Toda linha da tabela dá V — sem exceção.
Contradição	sempre F	$p \wedge \neg p$	Toda linha da tabela dá F — sem exceção.
Contingência	varia	$p \rightarrow q$	Existe pelo menos um V e pelo menos um F na coluna.

Guarde esta tabela — ela é a cola dos Exercícios 1 e 2, mais adiante.

CHEGA DE FAZER À MÃO



Verificando tudo com código

Montar uma tabela de 8 linhas na mão dá para fazer. De 16, já cansa. De 32, ninguém faz duas vezes sem errar. Em vez disso, geramos todas as combinações possíveis e avaliamos a fórmula em cada uma.

Toda combinação de V e F

Em C# escrevemos essa geração manualmente; em Python, o itertools já resolve isso pronto.

Combinacoes.cs



```
static IEnumerable<bool[]> Combinacoes(int n)
{
    for (int i = 0; i < (1 << n); i++)
    {
        var v = new bool[n];
        for (int j = 0; j < n; j++)
            v[j] = (i & (1 << j)) != 0;
        yield return v;
    }
}
```

combinacoes.py



```
from itertools import product

for valores in product([True, False], repeat=n):
    print(valores)
```

Ex.: $n = 2 \rightarrow (True, True), (True, False), (False, True), (False, False)$

Ideia: cada número de 0 a 2^n-1 , lido em binário, já é uma combinação de V/F.

De uma combinação para a tabela toda

Basta percorrer todas as combinações e avaliar a fórmula em cada uma.

TabelaVerdade.cs



```
static bool Implica(bool p, bool q) => !p || q;

static void TabelaVerdade(
    Func<bool[], bool> formula, int n)
{
    foreach (var v in Combinacoes(n))
        Console.WriteLine(
            $"{string.Join(" ", v)} -> {formula(v)}");
}

// (p ^ q) -> r
Func<bool[], bool> formula =
    v => Implica(v[0] && v[1], v[2]);
TabelaVerdade(formula, 3);
```

tabela_verdade.py



```
def implica(p, q):
    return (not p) or q

def tabela_verdade(formula, n_variaveis):
    for valores in product([True, False], repeat=n_variaveis):
        resultado = formula(*valores)
        print(valores, "->", resultado)

# (p ^ q) -> r
formula = lambda p, q, r: implica(p and q, r)
tabela_verdade(formula, 3)
```


Tautologia, contradição ou contingência?

Junte todos os resultados numa lista e verifique se são todos V, todos F, ou nem um nem outro.

Classificar.cs



```
static string Classificar(
    Func<bool[], bool> formula, int n)
{
    var r = Combinacoes(n).Select(formula).ToList();
    if (r.All(x => x)) return "tautologia";
    if (r.All(x => !x)) return "contradição";
    return "contingência";
}

Console.WriteLine(Classificar(v => v[0] || !v[0], 1));
Console.WriteLine(Classificar(v => v[0] && !v[0], 1));
Console.WriteLine(
    Classificar(v => Implica(v[0], v[1]), 2));
```

classificar.py



```
def classificar(formula, n_variaveis):
    combos = product([True, False], repeat=n_variaveis)
    resultados = [formula(*v) for v in combos]
    if all(resultados):
        return "tautologia"
    if not any(resultados):
        return "contradição"
    return "contingência"

print(classificar(lambda p: p or (not p), 1))
print(classificar(lambda p: p and (not p), 1))
print(classificar(lambda p, q: implica(p, q), 2))
```



Isso escala para qualquer número de variáveis

O mesmo Classificar / classificar funciona para 4, 5 ou 10 variáveis — só muda o n. É impraticável fazer isso à mão além de 3 ou 4 variáveis, mas é trivial em código: o computador não se importa se são 8 linhas ou 1024.

`2 variáveis → 4 linhas`

`5 variáveis → 32 linhas`

`10 variáveis → 1024 linhas`



Hora de praticar

Quatro exercícios: montar tabela, classificar, traduzir e um desafio de equivalência.



EXERCÍCIO 1

Montando à mão

Monte a tabela-verdade completa de $(p \rightarrow q) \wedge (q \rightarrow r)$ — 3 variáveis, 8 linhas. Preencha por colunas: primeiro $p \rightarrow q$, depois $q \rightarrow r$, por último a conjunção.

p	q	r	$p \rightarrow q$	$q \rightarrow r$	$(p \rightarrow q) \wedge (q \rightarrow r)$
V	V	V	?	?	?
V	V	F	?	?	?
V	F	V	?	?	?
V	F	F	?	?	?
F	V	V	?	?	?
F	V	F	?	?	?
F	F	V	?	?	?
F	F	F	?	?	?



GABARITO · EXERCÍCIO 1

Montando à mão

p	q	r	$p \rightarrow q$	$q \rightarrow r$	$(p \rightarrow q) \wedge (q \rightarrow r)$
V	V	V	V	V	V
V	V	F	V	F	F
V	F	V	F	V	F
V	F	F	F	V	F
F	V	V	V	V	V
F	V	F	V	F	F
F	F	V	V	V	V
F	F	F	V	V	V

Uma contingência: 4 linhas dão V, 4 dão F. Repare que quando p é F, $p \rightarrow q$ já é sempre V — só o resto decide.



EXERCÍCIO 2

Classificando

Classifique cada fórmula como tautologia, contradição ou contingência.

(a) $p \rightarrow (q \rightarrow p)$

(b) $(p \wedge q) \wedge \neg(p \vee q)$

(c) $(p \rightarrow q) \wedge q$



GABARITO · EXERCÍCIO 2

Classificando

$$p \rightarrow (q \rightarrow p)$$

(a)

Tautologia

Se p é V, a implicação já fecha em V. Se p é F, o condicional externo é vacuamente V.

$$(p \wedge q) \wedge \neg(p \vee q)$$

(b)

Contradição

Pede $p \wedge q$ verdadeiro e, ao mesmo tempo, $p \vee q$ falso — impossível simultaneamente.

$$(p \rightarrow q) \wedge q$$

(c)

Contingência

$V, V \rightarrow V$; $V, F \rightarrow F$; $F, V \rightarrow V$; $F, F \rightarrow F$ — mistura de V e F.



EXERCÍCIO 3

Traduzindo e verificando

“Se estudo e durmo bem, então passo.”

Modele como uma fórmula de 3 variáveis. Depois, use `tabela_verdade` / `TabelaVerdade` para encontrar uma linha em que a fórmula é falsa.

p =

q =

r =

Fórmula e linha falsa encontrada:



GABARITO · EXERCÍCIO 3

Traduzindo e verificando

p

“estudo”

q

“durmo bem”

r

“passo”

$$(p \wedge q) \rightarrow r$$

p	q	r	$(p \wedge q) \rightarrow r$
V	V	F	F

Única linha falsa: estudei e dormi bem, mas não passei — a promessa foi quebrada.



EXERCÍCIO 4 · DESAFIO

Fórmulas equivalentes

Escreva uma função `SaoEquivalentes(f, g, n)` / `sao_equivalentes(f, g, n)` que devolva `true/True` quando duas fórmulas de n variáveis produzirem a mesma coluna de resultados. Use-a para confirmar que $\neg(p \wedge q)$ e $\neg p \vee \neg q$ são equivalentes.

dica: compare $formula(v)$ com $f(v)$ e $g(v)$ para toda combinação v gerada por $Combinacoes(n)$

 `sao_equivalentes.py`

```
def sao_equivalentes(f, g, n):  
    return _____
```

Teste: `sao_equivalentes(lambda p,q: not(p and q), lambda p,q: (not p) or (not q), 2)` deve dar `True`.



GABARITO · EXERCÍCIO 4

Fórmulas equivalentes

 `sao_equivalentes.py`



```
def sao_equivalentes(f, g, n):  
    combos = product([True, False], repeat=n)  
    return all(f(*v) == g(*v) for v in combos)
```

Equivalente em C#: `SaoEquivalentes(f, g, n) => Combinacoes(n).All(v => f(v) == g(v));`

p	q	$\neg(p \wedge q)$	$\neg p \vee \neg q$
V	V	F	F
V	F	V	V
F	V	V	V
F	F	V	V

Onde estamos no semestre



Depois de saber montar e classificar tabelas, a Aula 06 troca a tabela por manipulação simbólica.

Referências

Livros — teoria



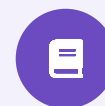
ROSEN, K. H.

Matemática Discreta e suas Aplicações. 7. ed.
AMGH/McGraw-Hill — cap. Lógica Proposicional
(tautologias, contradições, contingências).



GERSTING, J. L.

Fundamentos Matemáticos para a Ciência da
Computação. LTC — cap. Lógica.



SCHEINERMAN, E. R.

Matemática Discreta: uma introdução. Cengage
— seção sobre tabelas-verdade e classificação
de fórmulas.

Documentação e vídeos



Python — combinações e agregação

[itertools.product](#) — gera todas as combinações de valores.

[all\(\) e any\(\)](#) — a base de classificar().



Vídeos

[Exercícios resolvidos de tabela-verdade](#) — tautologia, contradição, contingência.

[Leis de De Morgan](#) — explicadas com tabela-verdade.



PRÓXIMA PARADA

Álgebra proposicional (equivalências lógicas)

A mesma De Morgan de hoje, agora provada sem tabela nenhuma.

Guarde a tabela-mestra de tautologia/contradição/contingência — ela continua valendo o semestre inteiro.